

Du C au C++ moderne

Surcharge, bibliothèque standard et gestion des ressources

Stéphane Abide Raphaël Granger

Université Côte d'Azur – LJAD

Cours : 1 heure – TD/TP : 2 heures

Où en sommes-nous ?

Au cours précédent

Nous avons appris à lire une signature :

```
T           // recevoir une valeur
T&          // modifier l'objet reçu
const T&     // lire sans modifier
```

La signature exprime comment la fonction reçoit et peut utiliser ses arguments.

Notre `IntArray` reste toutefois proche du C : sa mémoire est encore gérée manuellement.

Aujourd'hui : quatre directions

- ① un même nom de fonction, plusieurs signatures ;
- ② `std::string` et `std::vector` gèrent leur stockage ;
- ③ `nullptr` exprime explicitement un pointeur nul ;
- ④ les opérations se rapprochent progressivement des données.

Comment empêcher un objet d'être placé dans un état incohérent et automatiser la gestion de ses ressources ?

Surcharge : un même nom pour plusieurs fonctions

```
double norm(double x)
{
    return std::abs(x);
}

double norm(double x, double y)
{
    return std::hypot(x, y);
}

double a{norm(-3.0)};
double b{norm(3.0, 4.0)};
```

`norm(-3.0)`

Un argument : le compilateur choisit `norm(double)`.

`norm(3.0, 4.0)`

Deux arguments : il choisit `norm(double, double)`.

Même nom, signatures différentes.

Le type de retour seul ne suffit pas pour distinguer deux fonctions.

En C, nous aurions normalement choisi deux noms différents.
En C++, les arguments permettent au compilateur de choisir.

Résoudre un appel : trouver la meilleure surcharge

```
void evaluate(int order);  
void evaluate(double time);  
  
evaluate(4);           // exacte : int  
evaluate(0.25);        // exacte : double  
evaluate('a');         // conversion en int  
  
void scale(long);  
void scale(double);  
scale(2);              // erreur : appel ambigu
```

appel : `evaluate(4)`



Résolution du nom
trouver les fonctions
nommées `evaluate`



garder les surcharges
compatibles avec
les arguments



choisir la meilleure
correspondance

Question

Si aucune surcharge n'est meilleure que les autres, que doit faire le compilateur ?

Réponse

Erreur de compilation : l'appel est ambigu.

std::string : une chaîne devient un objet

Acquis en C

```
char label[64];

strcpy(label, "pressure");
strcat(label, "_mean");

printf("%s\n", label);
```

Choix C++

```
std::string label{"pressure"};

label += "_mean";

std::cout << label << '\n';
std::cout << label.size() << '\n';
```

Chaîne C

- tableau de `char` terminé par `'\0'` ;
- taille du stockage à gérer ;
- fonctions séparées : `strlen`, `strcpy`, `strcat` ;
- dépassement possible si le stockage est trop petit.

std::string

- taille du stockage gérée par l'objet ;
- copie et affectation naturelles ;
- concaténation avec `+` ou `+=` ;
- taille par `.size()`, destruction automatique.

On manipule une valeur chaîne, pas un tableau et sa capacité.

std::string : copie, concaténation et comparaison

Copier puis modifier

```
std::string a{"pressure"};
std::string b{a};          // copie

b += "_mean";

std::cout << a << '\n'; // pressure
std::cout << b << '\n'; // pressure_mean
```

Question

Modifier b modifie-t-il aussi a ?

Comparer le contenu

```
/* C */
if (strcmp(left, right) == 0) {
    /* contenus égaux */
}
```

```
// C++
if (a == "pressure") {
    // contenu égal
}
```

Réponse

Non. `b` est une chaîne indépendante. Avec `std::string`, `==` compare directement le contenu.

`std::string` gère son stockage et se copie indépendamment.
La comparaison porte sur le contenu ; la destruction libère le stockage.

Interagir avec une API C : `c_str()`

L'API C attend

```
/* chaine terminee par '\0' */  
void print_label(const char* text);  
  
/* la fonction lit text */  
/* sans le modifier */
```

Le code C++ fournit

```
std::string label{"pressure_mean"};  
  
print_label(label.c_str());  
  
const char* p{label.c_str()};
```

Lire seulement

Le type renvoyé est `const char*`.

Ne pas libérer

Le pointeur appartient toujours à la string.

Après une modification

Redemander `c_str()` avant de réutiliser la vue C.

`c_str()` fournit une vue C de la chaîne.

Elle permet de transmettre une `std::string` à une API C.

Du pointeur nul du C à nullptr

Acquis en C

```
int* p = NULL;

/* historiquement possible */
int* q = 0;
```

NULL est une macro historique ; 0 reste avant tout une valeur entière.

```
void inspect(int);
void inspect(double*);

inspect(0);           // inspect(int) : zero entier
inspect(nullptr);    // inspect(double*) : pointeur nul
```

Question

Avec 0, parle-t-on du nombre zéro ou de l'absence de cible ?

Choix C++

```
double* p{nullptr};

if (p == nullptr) {
    // aucun objet designe
}
```

nullptr exprime directement qu'aucun objet n'est désigné.

Réponse

0 est un entier ; nullptr exprime explicitement l'absence de cible.

Allocation dynamique : malloc/free et new/delete

Acquis en C

```
double* a = malloc(n * sizeof *a);

/* reserve un nombre d'octets */

/* utilisation */
free(a);
```

malloc

Réserve des octets et renvoie un `void*`. Il ne connaît pas le type d'objet attendu.

Choix C++

```
double* a{new double[n]};

// cree n objets double

/* utilisation */
delete[] a;
```

new

Connaît le type, crée les objets et renvoie directement un pointeur du bon type.

`new/delete` sert ici à comprendre la gestion manuelle.
En C++ moderne, on préfère confier la ressource à un objet.

new/delete : faire correspondre les formes

Un objet

```
double* p{new double{3.14}};  
  
// utilisation de *p  
  
delete p;
```

Un tableau

```
double* a{new double[n]{} };  
  
// elements initialises a zero  
  
delete[] a;
```

Initialisation

`new double[n]{}` initialise les éléments du tableau à zéro.

Échec d'allocation

`malloc` renvoie `NULL`. Par défaut, `new` lance `std::bad_alloc`.

`new` $\longleftrightarrow$ `delete` `new[]` $\longleftrightarrow$ `delete[]`

La durée de vie d'une ressource peut être gérée par un objet

Gestion manuelle

```
double* a{new double[n]{};};  
a[0] = 3.5;  
// calculs...  
delete[] a;
```

Gestion automatique avec `std::vector`

```
std::vector<double> a(n, 0.0);  
a[0] = 3.5;  
// calculs...  
// aucune libération explicite
```

création de `a` $\longrightarrow$ utilisation du stockage $\longrightarrow$ disparition de `a` $\longrightarrow$ libération automatique

La ressource suit la durée de vie de l'objet : première idée du RAII.
`std::vector` prend ici en charge le stockage à notre place.

Compilation séparée en C++ : .hpp et .cpp

statistics.hpp

```
#pragma once
#include <vector>

double mean(const std::vector<double>& x);
void center(std::vector<double>& x);
```

Déclarations visibles par l'utilisateur du module.

statistics.cpp

```
#include "statistics.hpp"

double mean(const std::vector<double>& x)
{
    if (x.empty()) return 0.0;
    double sum{};
    for (double value : x) sum += value;
    return sum / x.size();
}
```

Définitions et détails de l'implémentation.

```
g++ -std=c++20 -Wall -Wextra -pedantic -c statistics.cpp
g++ -std=c++20 -Wall -Wextra -pedantic -c main.cpp
g++ statistics.o main.o -o statistics
```

Dans un header, éviter `using namespace std;` : ce choix serait imposé à tous les fichiers qui l'incluent.

Même principe qu'en C : déclarations dans le `.hpp`, définitions dans le `.cpp`.
Les signatures contiennent maintenant références, `const`, surcharges et types de la bibliothèque.

Fil rouge : rapprocher les opérations des données

Fonctions libres

```
void push_back(IntArray&, int);  
int at(const IntArray&, int);  
  
push_back(values, 7);
```

Fonctions membres

```
struct IntArray {  
    // representation...  
  
    void push_back(int);  
    int at(int) const;  
};  
  
values.push_back(7);
```

La même définition, deux objets différents

```
a.push_back(7);  
b.push_back(12);
```

L'algorithme ne change pas nécessairement.
Avec `values.push_back(7)`, l'objet concerné apparaît directement dans l'appel.

Des fonctions membres ne suffisent pas

La représentation reste publique

```
struct IntArray {  
    int capacity{};  
    int size{};  
    int* data{nullptr};  
    void push_back(int);  
};
```

Invariant : règle qui doit rester vraie pour qu'un objet soit dans un état valide.

$$0 \leq \text{size} \leq \text{capacity}$$

$\text{capacity} = 0 \Rightarrow \text{data} = \text{nullptr}$

$\text{capacity} > 0 \Rightarrow \text{data}$ désigne une zone pouvant contenir **capacity** entiers.

```
values.size = -12;
```

$\text{size} < 0$

```
values.size = 10;  
values.capacity = 2;
```

$\text{size} > \text{capacity}$

```
values.capacity = 2;  
values.data = nullptr;
```

capacité non nulle sans stockage

Les champs publics permettent de contourner les opérations qui préservent l'invariant.

La prochaine étape consiste à contrôler l'accès : encapsulation.

Et qui libère la ressource ?

```
IntArray make_array(int n)
{
    IntArray a{};
    a.data = new int[n]{};
    a.capacity = n;
    a.size = 0;
    return a;
}
```

```
IntArray a = make_array(10);
IntArray b = a;
```

- Qui appelle `delete[]` ?
- Que se passe-t-il en cas d'oubli ?
- Que signifie copier `data` ?

La copie implicite recopie la valeur du pointeur : les deux objets désignent alors la même zone mémoire. **Copier le pointeur ne copie pas les données pointées.**

Question de conception

Un type qui possède une ressource doit définir ce qui se passe lors de sa création, de sa copie et de sa destruction.

Nous ne résolvons pas encore le problème : il motive le prochain chapitre.

Pourquoi avons-nous maintenant besoin des classes ?

Problème 1 — état invalide

```
values.size = -12;
```

Empêcher les modifications directes qui cassent l'invariant.

Problème 2 — durée de vie

```
values.data = new int[n];
```

Garantir l'acquisition et la libération de la ressource.

encapsulation

constructeur / destructeur

objet responsable de son invariant et de sa ressource

La ressource suit la durée de vie de l'objet.

RAII : *Resource Acquisition Is Initialization.*

La ressource est acquise à la construction et libérée à la destruction.